UD 01 · AMPLIACIÓN PRÁCTICA 05

Python para programadores Java y JavaScript

Sintaxis esencial antes de trabajar con Odoo en profundidad.

- if · for · while
- listas · dict · funciones
- ficheros · clases

Python para quien ya sabe programar

Este material no vuelve a explicar qué es una variable, un bucle o una función. El objetivo es que aprendas cómo se escriben en Python ideas que ya conoces de Java y JavaScript. Después practicarás cada bloque en las relaciones 05 a 11.

Idea clave

En Python la sangría forma parte de la sintaxis. No se usan llaves para delimitar bloques y normalmente no se escribe punto y coma al final de cada línea.

1. Bloques, comentarios y nombres

```
# Comentario
precio_unitario = 12.50
nombre_cliente = "Lucía"

if precio_unitario > 10:
    print("Precio superior a 10")
```

Usa cuatro espacios para cada nivel de sangría. Los nombres de variables y funciones suelen escribirse en snake_case. True, False y None empiezan con mayúscula.

2. Condicionales: if, elif y else

```
stock = 3

if stock == 0:
    print("Agotado")
elif stock < 5:
    print("Stock bajo")
else:
    print("Stock suficiente")
```

Python usa elif donde en Java o JavaScript escribirías else if. Los operadores lógicos son and, or y not. Para comprobar pertenencia existe in.

3. for y range()

```
for i in range(5):
    print(i)          # 0, 1, 2, 3, 4

for i in range(1, 6):
    print(i)          # 1, 2, 3, 4, 5

productos = ["Teclado", "Ratón", "Monitor"]
for producto in productos:
    print(producto)
```

No existe el for clásico for (i = 0; i < n; i++). Para recorrer secuencias se usa for directamente; para generar números se usa range().

4. while con condición explícita

```
opcion = -1

while opcion != 0:
    opcion = int(input("Opción (0 para salir): "))
    print(f"Has elegido {opcion}")
```

La condición se comprueba antes de cada vuelta. En estas prácticas evita construir menús con while True y break: expresa directamente la condición de continuación.

5. Listas, tuplas, conjuntos y diccionarios

```
productos = ["Teclado", "Ratón"]
productos.append("Monitor")

coordenadas = (10, 20)
categorias = {"Hardware", "Servicios"}

cliente = {
    "nombre": "Lucía",
    "ciudad": "Granada"
}
print(cliente["nombre"])
```

Una list es mutable y ordenada; una tuple suele representar datos que no quieres modificar; un set evita duplicados; un dict almacena pares clave-valor.

6. Cadenas y operaciones frecuentes

```
texto = "  Granada,ES  "
limpio = texto.strip()
minusculas = limpio.lower()
partes = limpio.split(",")

print(limpio[0:7])
print("granada" in minusculas)
```

strip() elimina espacios al principio y al final; lower()/upper() cambian mayúsculas; split() separa una cadena; los slices usan [inicio:fin].

7. Funciones: def, parámetros y return

```
def calcular_subtotal(precio, cantidad):
    subtotal = precio * cantidad
    return subtotal

resultado = calcular_subtotal(25.0, 3)
print(resultado)
```

Las funciones se declaran con def. Si no ejecutan return, devuelven None. Puedes usar parámetros con valor por defecto y argumentos por nombre.

8. Parámetros por defecto y anotaciones de tipo

```
def aplicar_descuento(importe: float, porcentaje: float = 0) -> float:
    descuento = importe * porcentaje / 100
    return importe - descuento

print(aplicar_descuento(100))
print(aplicar_descuento(100, porcentaje=10))
```

Las anotaciones de tipo ayudan a leer el código y a los editores, pero Python no las convierte en una comprobación rígida como Java.

9. Excepciones y validación de entrada

```
dato_valido = False
precio = 0.0

while not dato_valido:
    try:
        precio = float(input("Precio: "))
        dato_valido = precio >= 0
        if not dato_valido:
            print("El precio no puede ser negativo")
    except ValueError:
        print("Introduce un número válido")
```

try/except permite reaccionar a errores previstos. Captura excepciones concretas, por ejemplo ValueError, en lugar de usar except sin especificar el tipo.

10. Ficheros y with open()

```
with open("clientes.txt", "w", encoding="utf-8") as archivo:
    archivo.write("Lucía;Granada\n")

with open("clientes.txt", "r", encoding="utf-8") as archivo:
    for linea in archivo:
        print(linea.strip())
```

with cierra el fichero automáticamente incluso si ocurre un error. Indica encoding="utf-8" en los ficheros de texto de estas prácticas.

11. Módulos e importaciones

```
# calculos.py
def calcular_iva(base, porcentaje):
    return base * porcentaje / 100

# main.py
from calculos import calcular_iva

print(calcular_iva(100, 21))
```

Un archivo .py puede actuar como módulo. Separar funciones reutilizables ayuda a organizar programas mayores.

12. Clases, objetos y self

```
class Producto:
    def __init__(self, nombre, precio):
        self.nombre = nombre
        self.precio = precio

    def mostrar(self):
        print(f"{self.nombre}: {self.precio:.2f} EUR")

producto = Producto("Teclado", 25.0)
producto.mostrar()
```

self hace referencia al objeto actual. __init__ inicializa el objeto. Esta sintaxis será especialmente útil más adelante porque los modelos de Odoo se definen mediante clases de Python.

13. Herencia

```
class Empleado:
    def __init__(self, nombre):
        self.nombre = nombre

class Comercial(Empleado):
    def __init__(self, nombre, ventas):
        super().__init__(nombre)
        self.ventas = ventas
```

La clase hija escribe el nombre de la clase base entre paréntesis. super() permite reutilizar el constructor de la clase padre.

14. Herramientas muy útiles

```
precios = [10, 20, 30]

print(len(precios))
print(sum(precios))
print(min(precios))
print(max(precios))

for posicion, precio in enumerate(precios, start=1):
    print(posicion, precio)

dobles = [precio * 2 for precio in precios]
```

len(), sum(), min(), max(), enumerate() y las comprensiones de listas aparecen con frecuencia en código Python. Úsalas cuando hagan el código más claro, no por obligación.

15. Un programa organizado

```
def main():
    print("Inicio del programa")


if __name__ == "__main__":
    main()
```

Este patrón hace explícito el punto de entrada del programa. No es obligatorio para scripts pequeños, pero resulta útil cuando empiezas a separar el código en funciones y módulos.

Orden de práctica recomendado

Relación	Contenido
05	Condicionales y validación
06	Bucles for, while y range
07	Colecciones y cadenas
08	Funciones y descomposición
09	Excepciones, ficheros y módulos
10	Programación orientada a objetos
11	Mini-proyectos integradores